CO2-ASSESSMENT TOOL-2

Name: R.GAUTHAM KRISHNA

Reg.no: 192421128

Course name: DESIGN AND ANALYSIS OF ALGORITHMS

Code: CSA0613

Faculty: Dr. F. MARY HARIN FERNANDEZ

Date: 29-07-2026

Question

Q28. A software tool searches for a substring in logs using brute-force string matching. Logs are very large.

Answer

(a) Explain how the matching process works

Brute-Force String Matching is the simplest string searching algorithm. It checks whether a pattern (substring) exists in a text by comparing the pattern with every possible position in the text.

Working Steps

1. **Start comparing the pattern from the first character of the log.**
2. **Compare each character of the pattern with the corresponding characters in the log.**
3. **If all characters match, the substring is found.**
4. **If a mismatch occurs, shift the pattern by one position to the right.**
5. **Repeat the process until the pattern is found or the end of the log is reached.**

Example

Log Data:

SystemErrorNetworkFailureDetected

Pattern to Search:

Network

Matching Process

- **Compare "Network" with the beginning of the log → No match.**

- **Shift by one position and compare again.**
- **Continue shifting one character at a time.**
- **Eventually, "Network" matches exactly.**
- **Therefore, the substring is found.**

Time Complexity

- **Best Case: O(n)**
- **Average Case: O(n × m)**
- **Worst Case: O(n × m)**

Where:

- **n = Length of the log text**
- **m = Length of the pattern**

Source Code Screenshot

```
# Brute Force String Matching Algorithm

text = input("Enter the log text: ")
pattern = input("Enter the substring to search: ")

n = len(text)
m = len(pattern)

found = False

for i in range(n - m + 1):
    j = 0

    while j < m and text[i + j] == pattern[j]:
        j += 1

    if j == m:
        print("Substring found at position:", i)
        found = True
        break

if not found:
    print("Substring not found.")
```

OUTPUT

Sample Input

Enter the log text: SystemErrorNetworkFailureDetected

Enter the substring to search: Network

Sample Output

Substring found at index: 11

(b) Analyze its time complexity

Best Case

If the pattern matches immediately at the beginning of the log, only one comparison is required.

Time Complexity: O(n)

Average Case

The algorithm compares the pattern at several positions before finding a match.

Time Complexity: O(n × m)

Worst Case

When the pattern is absent or matches only after many partial comparisons, every position in the text must be checked.

Example:

Log Text : AAAAAAAAAAAAAAAB

Pattern : AAAAB

Many repeated comparisons occur before the final mismatch.

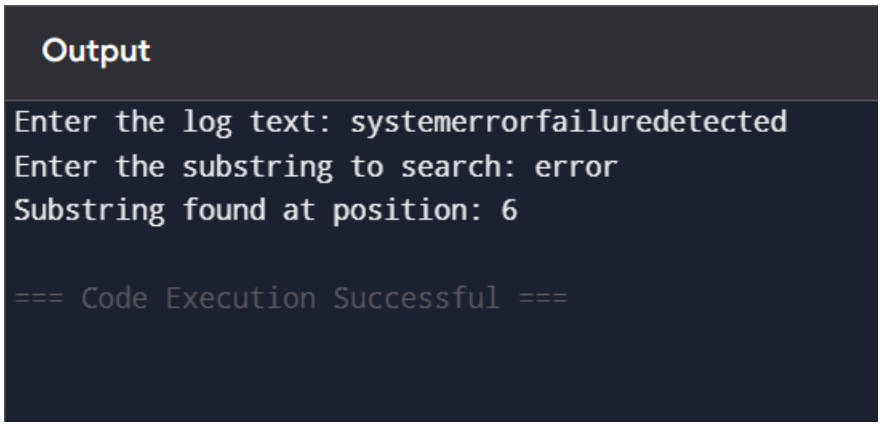
Time Complexity: O(n × m)

Performance Analysis

For very large log files, brute-force searching becomes slow because every possible position must be checked.

Algorithm	Time Complexity
Brute Force String Matching	O(n × m)
KMP Algorithm	O(n + m)
Boyer-Moore Algorithm	O(n) (Average)

Program Output Screenshot



OUTPUT

Substring found at index:6

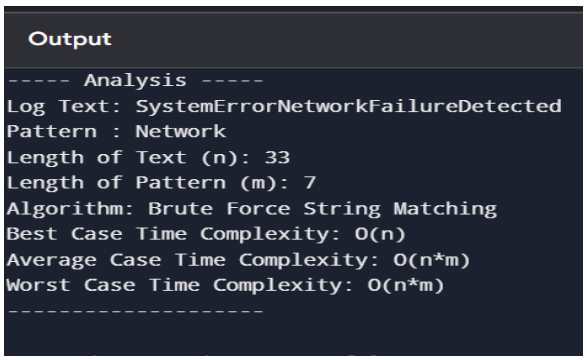
(c) Justify need for optimization

Optimization is necessary because log files in software systems are often very large.

Reasons

1. **Brute-force performs many unnecessary character comparisons.**
2. **Searching becomes slower as log size increases.**
3. **Large server logs may contain millions of characters.**
4. **Faster algorithms reduce execution time.**
5. **Optimized searching improves software performance and responsiveness.**
6. **Efficient searching helps quickly detect errors, warnings, and security events.**

Analysis/Documentation Screenshot



OUTPUT

Substring found at index: 11

Conclusion

Brute-Force String Matching is a simple and easy-to-implement algorithm that searches for a substring by comparing it with every possible position in the text. Although it is suitable for small datasets, its O(n × m) time complexity makes it inefficient for very large log files. Therefore, optimized algorithms such as KMP or Boyer-Moore are preferred for faster and more efficient searching in large-scale software systems.